

Introduction to Lists and Tuples

Christopher Martin - *Bowdoin College* 
c.martin@bowdoin.edu 

String Slices

We have seen that strings are a little different from the other **primitive** types...

They store multiple characters that we can access into using **positions** or **indices** ...

C	h	r	i	s	t	o	p	h	e	r
0	1	2	3	4	5	6	7	8	9	10

```
1 name: str = "Christopher"
2
3 print(name[0], name[5:9]) # Access position 0 and access from 5 up until 9
```

C toph

String Methods

We have also seen how we can use the `my_str.method()` syntax to ask questions and execute various behaviors on specific strings in using **methods** !

```
1 name: str = "Christopher Martin"
2 search: str = "r"
3
4 print(f"{name} in all uppercase is {name.upper()}! Was it originally all upper? {name.isupper()}")
5 print(f"{name} in all lowercase is {name.lower()}! Was it originally all upper? {name.islower()}")
6 print(f"There are {name.count(search)} {search}'s in {name}")
7 if search in name:
8     print(f"The first {search} is at position {name.find(search)}")
9
10 number_or_text: str = "54"
11 print(f"Is {number_or_text} all digits? {number_or_text.isdigit()}")
12 print(f"Is {number_or_text} all letters? {number_or_text.isalpha()}")
```

```
Christopher Martin in all uppercase is CHRISTOPHER MARTIN! Was it originally all upper? False
Christopher Martin in all lowercase is christopher martin! Was it originally all upper? False
There are 3 r's in Christopher Martin
The first r is at position 2
Is 54 all digits? True
```

Lists and Tuples

Lists and **Tuples** work in the same way but they allow us to group any combination of things together in one variable!

- Lists are created by surrounding a comma separated sequence with brackets `[]` !
- Tuples are created by surrounding a sequence with parentheses `()` !

```
1 numbers: list[int] = [1, 3, 3, 7]
2 letters: tuple[str] = ("a", "c", "b", "a", "z")
3
4 print(numbers)
5 print(letters)
```

```
[1, 3, 3, 7]
('a', 'c', 'b', 'a', 'z')
```

Lists and Tuple Ordering

Lists and Tuples store things in an ordered fashion just like strings!

We can access individual items with slices and determine their length!

[1 ,	3 ,	3 ,	7]	("a" ,	"c" ,	"b" ,	"a" ,	"z")
0	1	2	3	0	1	2	3	4

```
1 numbers: list[int] = [1, 3, 3, 7]
2 letters: tuple[str] = ("a", "c", "b", "a", "z")
3
4 print(numbers[0], numbers[1:], ": numbers has", len(numbers), "elements!")
5 print(letters[1], letters[2:], ": letters has", len(letters), "elements!")
```

```
1 [3, 3, 7] : numbers has 4 elements!
c ('b', 'a', 'z') : letters has 5 elements!
```

List Operators

List support many of the same operators as strings!

```
1 list_1: list[int] = [1, 3, 7]
2 list_2: list[str] = ["two", "four", "six"]
3
4 # We can use the in operator to check if something is in a list
5 if 2 in list_1:
6     print("2 is in list_1!")
7 else:
8     print("2 is not in list_1!")
9
10 # We can compare lists together!
11 if list_1 == list_2:
12     print("These two lists contain the same elements in the same order!")
13 else:
14     print("Something is different about these two lists!")
15
16 # We can concatenate lists together into a new list!
17 print(list_1 + list_2)
```

```
2 is not in list_1!
Something is different about these two lists!
[1, 3, 7, 'two', 'four', 'six']
```

Tuple Operators

Tuples support the same operators too!

```
1 tuple_1: tuple[float] = (2.0, 4.0, 6.0)
2 tuple_2: tuple[int] = (7, 3, 1)
3
4 # We can use the in operator to check if something is in a tuple
5 if 2 in tuple_1:
6     print("2 is in tuple_1")
7 else:
8     print("2 is not in tuple_1!")
9
10 # We can compare tuple together!
11 if tuple_1 == tuple_2:
12     print("These two tuples contain the same elements in the same order!")
13 else:
14     print("Something is different about these two tuples!")
15
16 # We can concatenate tuple together into a new tuple!
17 print(tuple_1 + tuple_2)
```

```
2 is in tuple_1
Something is different about these two tuples!
(2.0, 4.0, 6.0, 7, 3, 1)
```

Operator Incompatibility

However, we can't always mix lists and tuples together...

```
1 my_list: list[float] = [2.0, 4.0]
2 my_tuple: tuple[float] = (2.0, 4.0)
3
4 # We can compare lists and tuples but they will never be equal!
5 if my_list == my_tuple:
6     print("This can never happen 😭")
7 else:
8     print("One being a list and the other being a tuple makes them 'not equal'!")
9
10 # We can't concatenate lists and tuples together!
11 print(my_list + my_tuple)
```

Traceback (most recent call last):

```
File "/app/output.s", line 11, in <module>
    print(my_list + my_tuple)
          ~~~~~^~~~~~
```

TypeError: can only concatenate list (not "tuple") to list

Wait... these seem really similar...
Why do we have both?



Lists vs Tuples

Lists

Lists are **mutable** collections

They can be modified after creation!

```
1 my_list: list[int] = [1, 2, 3]
2
3 my_list[2] = 1
4 print(my_list)
```

```
[1, 2, 1]
```

Tuples

Tuples are **immutable** collections

They cannot be modified after creation!

```
1 my_tuple: tuple[int] = (1, 2, 3)
2
3 my_tuple[2] = 1
4 print(my_tuple)
```

```
Traceback (most recent call last):
  File "/app/output.s", line 3, in <module>
    my_tuple[2] = 1
    ~~~~~^~~~~
TypeError: 'tuple' object does not support item
assignment
```

List Methods - Mutable

Most of these methods do not return anything!

Method	Description
<code>my_list.append(v)</code>	Adds the given value <code>v</code> to the end of <code>my_list</code>
<code>my_list.insert(i, v)</code>	Adds the given value <code>v</code> to <code>my_list</code> at the index <code>i</code>
<code>my_list.remove(v)</code>	Removes the first occurrence of the value <code>v</code> from <code>my_list</code>
<code>my_list.pop(i)</code>	Removes <u>and returns</u> the element at the index <code>i</code> from <code>my_list</code>
<code>my_list.clear()</code>	Removes everything from <code>my_list</code>
<code>my_list.reverse()</code>	Reverses the order of the elements in <code>my_list</code>

Tuple Methods and additional List methods

Tuples have some methods as well but nearly as many!

- Lists also have these methods too!

Method	Description
<code>list_or_tuple.index(v)</code>	Returns the index of the first occurrence of the value <code>v</code> in <code>list_or_tuple</code>
<code>list_or_tuple.count(v)</code>	Returns the number of occurrences of the value <code>v</code> in <code>list_or_tuple</code>